# Eventify: Modern Microservices Platform

Carlos Andrés Abella
Daniel Felipe Paez
Leidy Marcela Morales

# CONTENTS

01

# PART 01

## Platform Overview

# Eventify: Comprehensive Event Management

Eventify is a full-stack platform built with a microservices architecture . It enables users to browse events, purchase tickets, and manage bookings, while providing organizers with tools to create and manage events efficiently. This project demonstrates modern software engineering practices including containerization, CI/CD pipelines, and full-stack development with multiple programming languages.

# Core Features and Capabilities

## Comprehensive Event Browsing

Search and filter events across multiple categories with real-time availability.

## Secure User System

Secure registration, login, and profile management with role-based access.

## Ticket & Order Management

Intuitive ticket selection, shopping cart, and a streamlined checkout process.

02

# PART 02

## Architecture Design

# Microservices Architecture Pattern

## React Frontend

User Interface Layer

## Java Backend

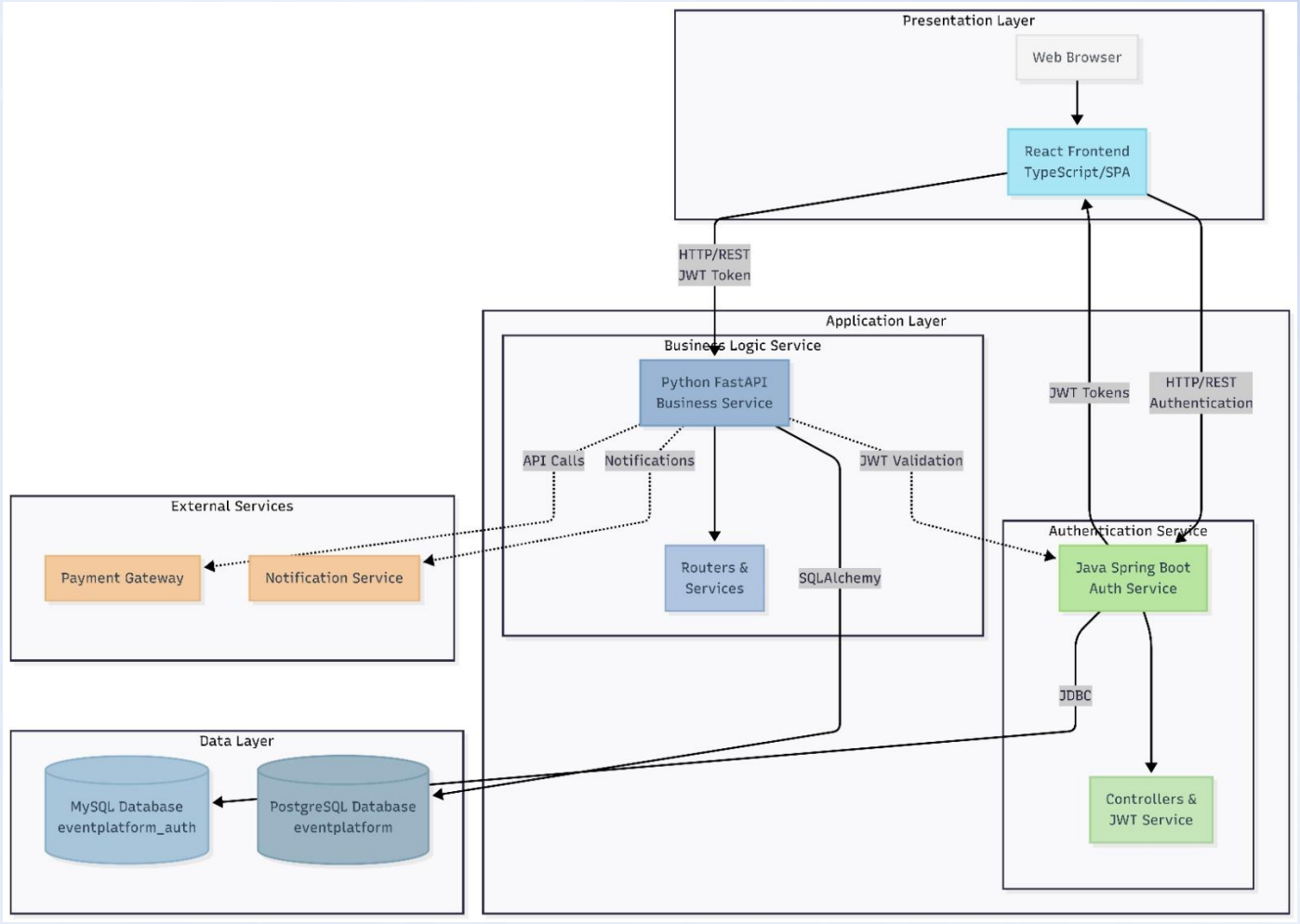
Authentication & Authorization

## Python Backend

Event Management & Business Logic

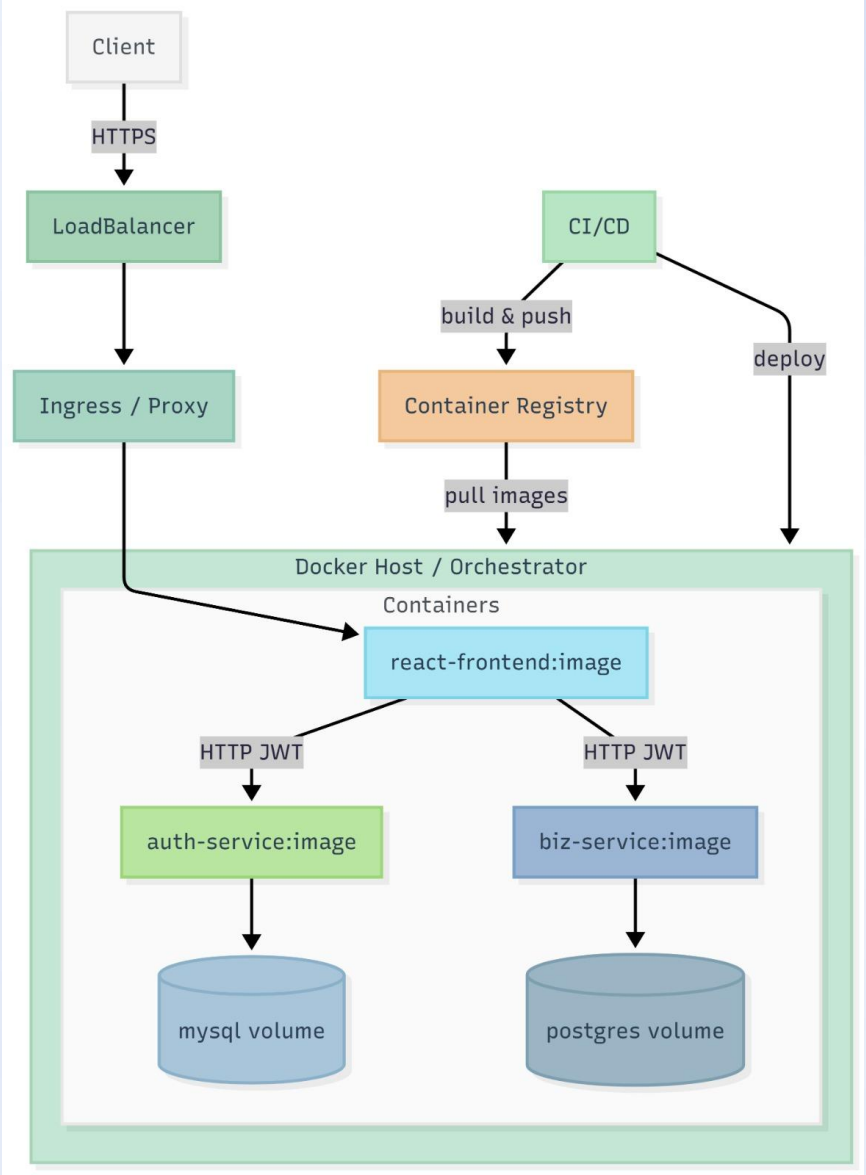
Three independent services communicating via RESTful APIs with shared JWT authentication, enabling independent scaling and technology optimization.

# Software Architecture




The shared JWT secret enables stateless authentication, allowing the Python backend to validate tokens issued by the Java backend without direct service-to-service calls.

# System Architecture



03

# PART 03

## Technology Stack

# Frontend Technologies

A modern, responsive web application built with a focus on developer experience and maintainability.

- **React 18 & TypeScript:** Type-safe component development.
- **Tailwind CSS:** Rapid, utility-first styling.
- **Redux Toolkit:** Centralized, predictable state management.
- **React Router:** Declarative client-side navigation.

# Java Backend Stack

Spring Boot 3.x

### Spring Security + JWT
Industry-standard authentication and authorization.

### Spring Data JPA
Simplifies database operations with MySQL 8.0.

### RBAC System
Role-Based Access Control for Admin, Organizer, and Buyer.

# Python Backend Stack

High-performance async operations for event management and business logic.

- **FastAPI & Python:** High-performance async web framework.
- **PostgreSQL 17:** Robust data management with JSONB support.
- **SQLAlchemy & Alembic:** Async ORM and database migrations.
- **Pydantic:** Automatic data validation using Python type hints.

04

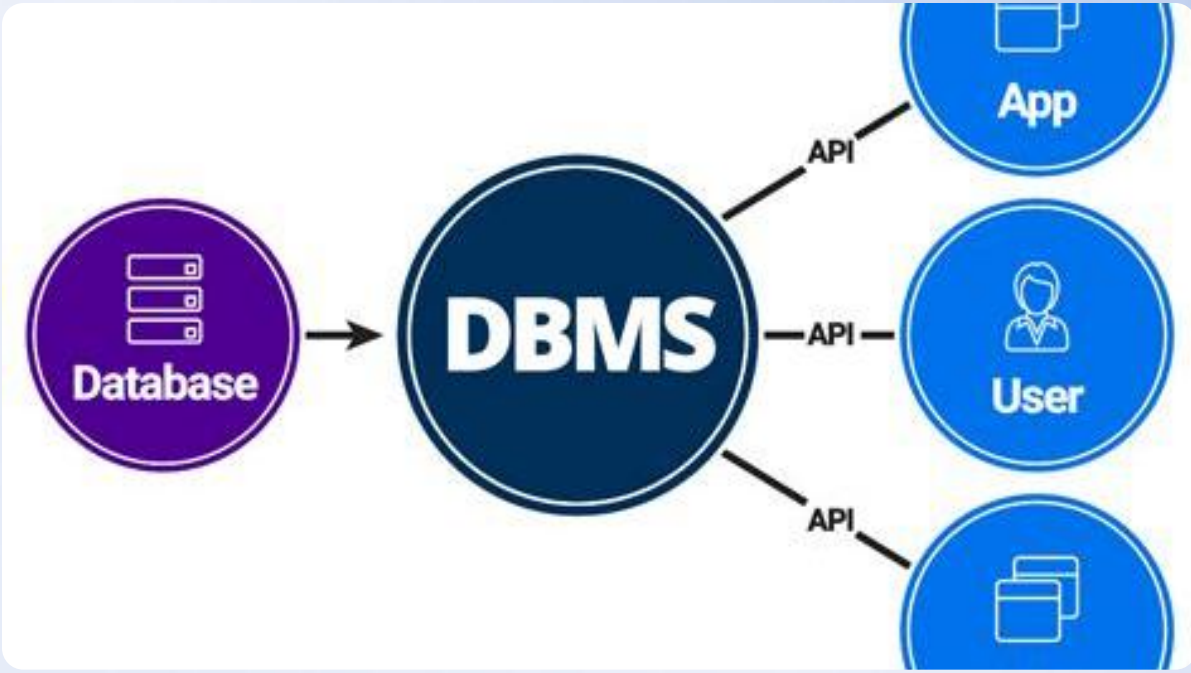
# PART 04

## Database Design

# MySQL Authentication Schema

The authentication database uses Single Table Inheritance for user types, ensuring performance and simplicity. It stores bcrypt-hashed passwords and implements role-based permissions for secure access control.

- BCrypt password hashing (strength 12)
- Role-Based Access Control (RBAC)
- Single Table Inheritance for users



# PostgreSQL Event Schema

A comprehensive schema designed for complex event management, featuring 31+ pre-loaded events across various categories.

### Events
Core event details

### Categories
Event classification

### Locations
Venue management

### Tickets
Inventory & pricing

### Orders
Purchase records

### Audit Logs
Data change tracking

05

# PART 05

## Security Implementation

# JWT Authentication Flow

### 1. User Login

Credentials validated by Java backend. Token contains user ID, email, and role.

### 2. JWT Issued

### 3. Token Shared

Used for all subsequent API requests.

### 4. Validation

Both backends validate using the shared secret.

This stateless flow ensures secure, scalable authentication without server-side session storage.

# Security Best Practices

## Authentication

BCrypt password hashing (strength 12) and JWT tokens with expiration.

## Data Protection

SQL injection protection, XSS prevention, and CSRF protection for state-changing ops.

## Network Security

CORS restricted to known origins, HTTPS required in production, and secure headers.

## Secret Management

Environment variables for sensitive data with no hardcoded credentials.

06

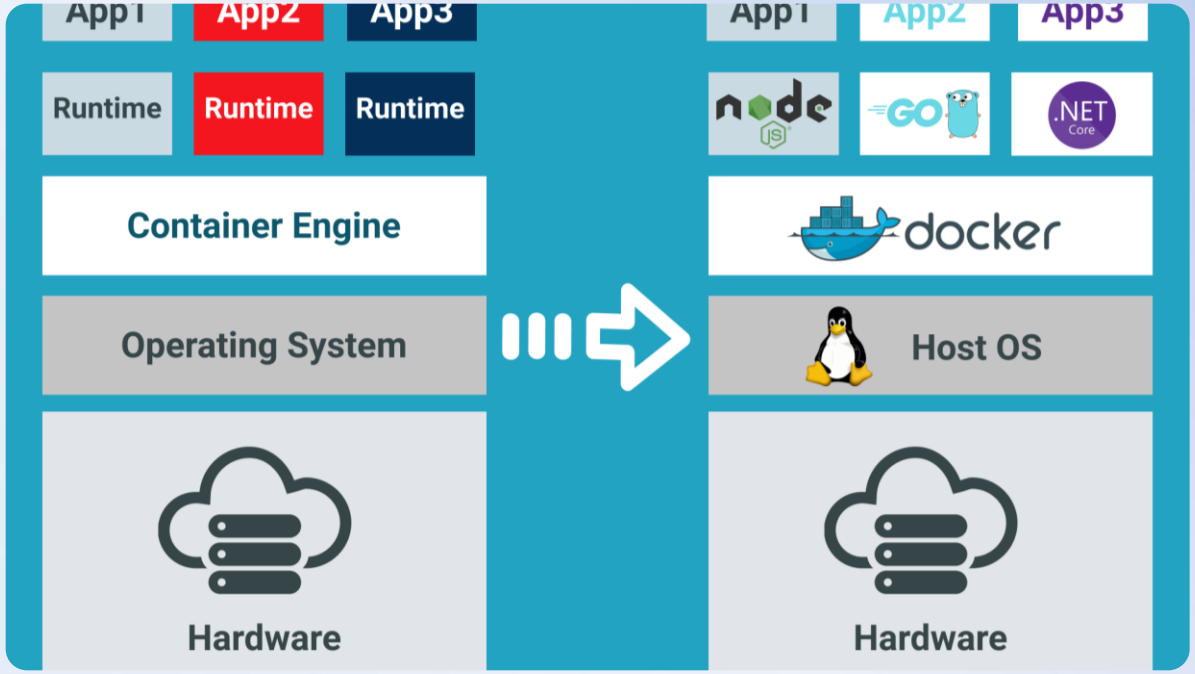
# PART 06

## Development Features

# Docker Containerization

Multi-stage Dockerfiles create optimized production images, separating build and runtime environments to minimize image size and attack surface.

- Optimized, production-ready images
- Consistent development environments
- Simplified deployment processes



# CI/CD Pipeline





1. Code Push
2. Automated Setup
3. Parallel Testing
4. Docker Build
5. Deployment

GitHub Actions automates the entire workflow, ensuring code quality and rapid delivery from commit to production.

07

# PART 07

## Testing Strategy

# Comprehensive Test Coverage

A multi-layered testing strategy ensures the reliability and stability of the entire platform, from individual units to integrated systems.

### Java Backend

Unit, integration, security, and repository tests with Maven and JUnit.

### Python Backend

Pytest suite with fixture-based setup, API testing, and >80% coverage.

# Frontend Testing Approach

## Component & Unit Testing

React Testing Library and Jest are used to test individual components and utility functions in isolation.

## Integration Testing

Tests user flows, such as authentication and checkout, to ensure different parts of the app work together.

## Snapshot Testing

Used to detect unintended UI changes by comparing the rendered component to a saved snapshot.

## Coverage Reporting

Comprehensive reports ensure critical parts of the application are well-tested before deployment.

08

# PART 08

## Performance Optimization

# Backend Performance

## Connection Pooling

HikariCP and SQLAlchemy async pools optimize database connections.

## Database Indexing

Indexes on foreign keys and frequently queried columns accelerate data retrieval.

## Async Operations

Non-blocking I/O throughout the Python backend for high concurrency.

## Compression & Caching

Gzip compression and HTTP caching headers improve response times.

# Frontend Optimization

## Code Splitting
Route-based lazy loading reduces initial bundle size.

## Tree Shaking
Webpack removes unused code for a leaner bundle.

## CSS Purging
Tailwind removes unused styles in production builds.

## Image Optimization
WebP format and lazy loading for faster page loads.

09

# PART 09

## Future Roadmap


# Immediate Enhancements

- Email verification & password reset functionality.
- Two-factor authentication (2FA) for enhanced security.
- OAuth2 integration (Google, Facebook).
- Event image uploads to cloud storage (S3/Azure).
- Payment gateway integration (Stripe/PayPal).
- Email notifications (SendGrid) for order confirmations.

# Long-term Vision

### Real-time Features
WebSocket for live notifications.

### Advanced Search
Elasticsearch integration.

### Mobile App
React Native application.

### ML & Analytics
Personalized recommendations.

Future developments focus on **scalability, user engagement, and intelligent features** to create a world-class event platform.

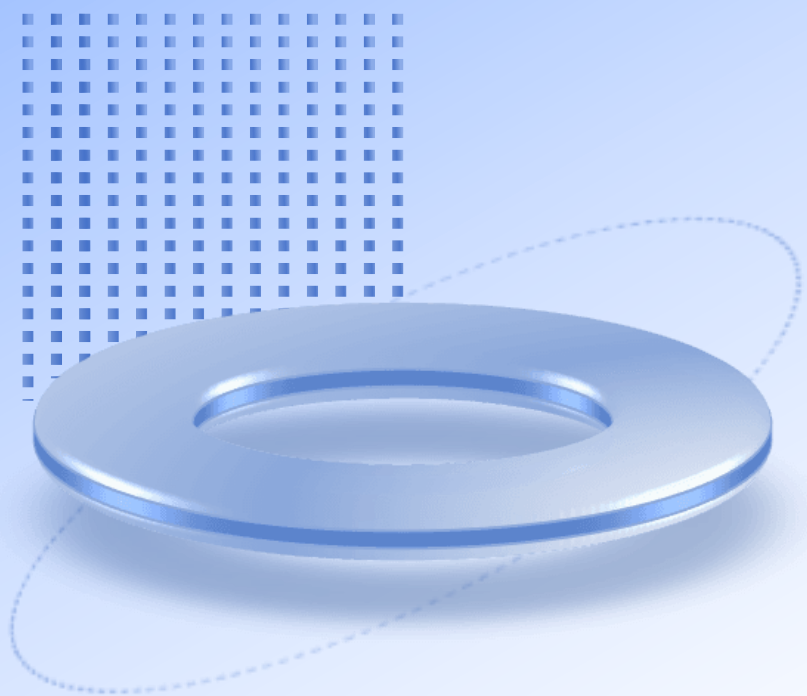
10

# PART 10

## Project Summary

# Educational Achievement

Eventify successfully demonstrates the practical application of modern software engineering principles. It showcases a complete, production-ready system built on a microservices architecture , integrating multiple programming languages, comprehensive testing, containerized deployment, and CI/CD automation. This project serves as a testament to full-stack development skills, from responsive UI design to secure backend APIs and database management, providing a solid foundation for real-world event management applications.



# THANKS